

Does Agentic Architecture Discovery Reproduce? An Audit Gate That Blocked Its Own Headline Result

Anonymous Author(s)

Abstract

Agents already *discover and exploit loopholes in microarchitectural simulators* [1], so we measure evaluation validity rather than design discovery. We release a CHIA audit loop that separates four variance sources an aggregate score hides (repeated-run, generation-seed, prompt and trace) and gates publication on a pre-registered threshold plus a double-blind annotation layer. Run in the official chia-champsim image, the loop *failed* its own gate: cross-seed CV 78.16% against a 5% threshold within one generated series, while repeated-run CV is 0.0000% over 45 runs, so the publish gate reads BLOCKED. Two candidates differing only in a struct name carry distinct source, design and binary digests yet bit-identical cycles: the seed moved the artifact, not the design. Annotation gives $\kappa = 0.859$ on specification diffs and 0.636 on real generated C++, both below our own 0.7 gate; on six measurement-grounded pairs two same-provider models never disagree yet each is right on only 3 of 6, and a second provider reproduces those three misses, all of them simulator escapes. Auditing that κ found the defect inside it: our own label generator read `#include` angle brackets as comparison operators and mislabelled three rename-only pairs, so the annotators we were scoring were correct and the ground truth was not. We report the negatives, ten defects found in our own automation, and the repairs instead of moving thresholds.

1 Introduction

Architecture 2.0 pipelines (ArchAgent [1], CHIA [2], QuArch) let agents propose and evaluate designs with no human in the loop, and the risk is not unclaimed: ArchAgent reports that its own pipeline found and exploited a loophole in a popular microarchitectural simulator, inflating reported performance without changing hardware logic [1]. Software-side evaluation shows the same fragility: the top three Terminal-Bench 2 submissions were all caught cheating via harness tricks [3], OpenAI dropped SWE-bench Verified after an audit found roughly 30% of SWE-bench Pro’s tasks broken [4], and an automated audit of 168 agent benchmarks flags over 25.7% of the evaluated *tasks* [5].

What is missing is an instrument, not another warning: a loop that runs against a real simulator, separates the variance sources an aggregate score hides, and refuses to publish when that separation fails. We contribute that loop plus what it said about us: a BLOCKED verdict on our own headline claim (§3.1), seeds moving artifacts without moving designs, trace-dependent reordering (§3.2), prompt response at 249.18% (§3.3), a failure mode text-reading auditors cannot catch and a second provider repeats (§3.5), and ten defects in our own automation (Table 1). Each was found by reading evidence, none by a green gate.

2 The Audit Loop

Prepare. Pin git SHA, interpreter and image digest; load the pre-registered configuration. Thresholds are frozen before any measured cell exists, which is what makes the §3 verdicts binding. **Run.** Execute a $|\text{seeds}| \times |\text{prompts}| \times |\text{traces}|$ grid, n repeats per cell, through CHIA’s ChampSimNode in the official image, recording the candidate digest, delivered binary digest and raw simulator JSON per cell. **Audit.** Two annotators classify candidate against reference using E0–E4: E0 equivalent, E1 parameter mislink, E2 direction error (LRU $\leftrightarrow$ MRU), E3 missing component, E4 added or fabricated component. The payload is built from an allowlist of id, design, reference, and the script refuses to run if any expected label survives. **Act.** Emit the scorecard and gate: publishable requires reproducibility PASS and audit PASS. A nullity check compares each candidate with a no-op reference measured on the same trace and instruction budget. The in-loop self-test and the independent-model layer are reported separately and never merged.

Setup. ghcr.io/ucb-bar/chia-champsim at digest sha256:610951d3. . . , ChampSim 164fdb1e. . . , Python 3.10.19, Ray 2.54.0; 46 unit tests pass in-tree and inside the pinned image. Grids ran on two c2d-standard-8 VMs in us-east1-b; on the three (design, trace) cells where they overlap they agree to the cycle. Three DPC-4 traces (fotonik3d, ligra BFSCC, imagick) with SHA-256 in results/grid_v6/provenance. 1M warmup + 4M simulated instructions, L2C. Candidates are real LLM-generated prefetcher modules retained with raw prompts and responses; seeds and prompts select the artifact and the simulator is never handed a seed. A separate 3×3 prompt-by-seed factorial exercises the prompt axis: **6 of 9** generated candidates compile, and all three failures abort within ≈ 13 s against ≈ 800 s for a cold success. **Calibration.** 5 gold + 5 adversarial specification cases and 4 semantic cases built from the generated C++ itself. Pre-registered: cross-seed CV < 5%; repeated-run CV, prompt spread, trace CV, top-1 stability and leave-one-trace-out Kendall τ reported; annotation gate $\kappa \geq 0.7$.

Release. A harness that only ever returns PASS cannot be distinguished from one that is not looking, so the artifact ships the blocked scorecard, raw cells and digests: loop, cases, generators, gate and scripts/independent_audit.py as CHIA-compatible blocks (BSD-3-Clause). The repository URL is filed in the submission form rather than printed here, so the paper stays anonymous. Every number quoted here is re-derived from the committed artifacts by scripts/verify_paper_claims.py (241 claims, non-zero exit on drift), The full audit report behind Table 1 ships in the same repository as paper/paper_extended.tex. Reproduce the headline grid with

```
python3 chia_loop/loops/audit_repro.py -version 6 -config
.tmp/cfg/grid_gcp.json -backend champsim_node -output-dir
/tmp/repro, passing an output directory outside results/ so a
replication cannot overwrite the committed evidence. Both compute
hosts and the sponsoring account were deleted on 2026-09-24.
```

3 Results

3.1 The gate blocked our own headline claim

Across 3 designs $\times$ 3 traces $\times$ 5 repeats (45 runs, on a second physical host) the loop reports cross-seed CV 78.16% against the 5% threshold, verdict NON-REPRODUCIBLE, publish gate BLOCKED. Repeated-run CV is 0.0000%: no cell varies across its five runs. The simulator is deterministic, and determinism is not reproducibility (the variance decomposition of Fig. 1 in the full report).

Cycles, IPC and instruction counts match the earlier run on all nine cells, yet 0 of 9 cells share a delivered binary digest: a rebuild on another host produces byte-different binaries while the simulated measurement stays bit-identical. A binary digest therefore evidences a distinct build, not design identity, and a gate demanding digest equality across hosts would reject a faithful reproduction. The converse failure is in our own screening data: module *names* are not identifiers either. `gen_fill_only_conservative_s2` occurs in both batches under different delivered binaries, measuring 1,132,568 and 1,129,110: a join keyed on the name silently drops one.

The seed axis explains the gap. `s1` and `s3` differ only in the struct name and two lines of continuation indentation, yet each carries a different source, design and binary digest while all three traces return bit-identical cycles. `s2` keeps a fill history, is 10.6% faster than the no-op reference on BFSCC, and reproduces the no-op cycle count exactly on imagick. The seed moved the artifact and not the design, and nullity is a per-trace property, so the evidence checker now separates `null_on_every_measured_trace` from trace-dependent inertness.

That 78.16% is scoped to one generated series, and a second series says why. A replication of the same prompt at seeds {2, 3} from a *different* candidate directory measures cross-seed CV 0.0000%, because the two directories hold different programs under the same module name (BFSCC cycles 6,420,195 versus 95,624,440). The 15 $\times$ gap is an input difference, not a measurement disagreement, and only the distinct `candidate_sha256` values made it checkable: a seed label is not a design identifier across series.

3.2 Trace choice reorders the candidates

Trace CV is 163.33%, top-1 stability 0.667, mean Kendall $\tau = 0.556$: the ranking is not stable under the trace subset that produced it. The control pair separates by 23.15% on BFSCC but 0.42% on fotonik3d, so the grid must be read per trace. And the stability number is not stable either: a second rectangle, 2 seeds $\times$ 2 prompts $\times$ 3 traces (12 cells, 36 runs), same harness, same pinned image, same traces, reports top-1 stability 1.0 and $\tau = 0.778$ over four designs. Nothing in the instrument changed; only the candidate set did. So $\tau = 0.556$ and $\tau = 0.778$ are both correct readings of the same code, and “the ranking is unstable” is a claim about a design set, not about agentic architecture discovery.

3.3 Prompt sensitivity is 249%, and the gate could not see it

A separate 1 seed $\times$ 3 prompt grid over three traces gives maximum prompt spread 249.18%, computed as $(\max - \min) / \text{mean}$; the spread is almost entirely one trace. The two axes do not mask

each other: the 2 $\times$ 2 rectangle measures cross-seed CV 23.50% and prompt spread 179.67% simultaneously, both far above the 5% gate.

The same grid carries a second, worse story. Because it held the seed constant, cross-seed CV was computed over a single value, and our variance helper returns 0.0 for fewer than two samples, so the scorecard printed cross-seed CV = 0.0000%, watched it satisfy the $< 5\%$ threshold, and returned REPRODUCIBLE/PUBLISHABLE. **An axis that never varied cannot fail a threshold on that axis**, and our gate could not tell a measured 0% from an unmeasured one. The fix counts factor levels: an axis with fewer than two levels is *unexercised*, can never satisfy the gate, and is named in `publish_blockers` as `axis_not_exercised:seed`. `scripts/rescore_grid.py` then rescored every retained `raw.json` without re-running the simulator; the verdict is NON-REPRODUCIBLE/BLOCKED.

3.4 Independent annotators, and a shared blind spot two providers reproduce

On the 10 specification cases the two models give Cohen’s $\kappa = 0.859$ (0.900 agreement; verdict accuracy 10/10, error-code exactness 7/10 and 8/10). Widening to pairs of the nine factorial candidates, built from real generated C++ rather than dictionary diffs, gave $\kappa = 0.7235$ over 36 pairs; Table 1 row 2 shows why that cannot be quoted. After relabeling, **33 of 36** pairs carry a derivable verdict, both annotators are correct on **all 33**, they never disagree ($\kappa = 1.0$ on verdict), and combined-label κ falls to **0.636**. That is below our own 0.7 gate too, so both external sets are red: 0.7235 was the only value that passed, and it passed on withdrawn labels. On the **six** measurement-grounded cases the two models do not disagree (0 disagreements), and each is right on only 3 of 6 (verdict accuracy 0.5); combined-label κ is **0.455**. The three cases they miss together are *exactly* the three escapes of §3.5, and no escape is missed by only one of them.

One caveat we cannot resolve: **all 33 labellable gold labels in that set are not_equivalent**, so a constant `not_equivalent` rater would also reach $\kappa = 1.0$, and our helper returns exactly 1.0 on degenerate marginals. The verdict-only row is therefore *not* evidence of reliability; the 0.636 is. And high verdict agreement is the warning, not the reassurance: κ measures correlation between annotators, not correctness against measurement. $\kappa = 1.0$ is what a shared blind spot looks like from the inside, and also what a shared *correct* reading looks like when the ground truth is the defective party.

3.5 A nullity escape both auditors missed, and a second provider repeats

Compile-and-run is not evidence of a design. Against a cold no-op reference (the nullity screen of Fig. 1b in the full report) measuring 1,138,748 cycles, two *distinct* binaries (a generated stride prefetcher and a repaired aggressive-offset candidate) return exactly 1,138,748, while every genuine prefetcher and the next-line control differ. A separate screening of the factorial added a third: `gen_aggressive_offset_s1`. Three nullities across two batches: any verifier accepting “builds, runs, produces numbers” passes all three.

Both annotators classified this case `not_equivalent/E4`; measurement says equivalent. They agree with each other and disagree with the simulator. Three of the six measured cases are nullity escapes, from two screening batches and three distinct binaries; both raters miss **all three** in the same direction and miss none of the other three. That is a systematic blind spot, not independent noise, on a denominator of six. Nor is it an artefact of framing: swapping candidate and reference leaves all twelve verdicts unchanged (0/12 flips, `scripts/annotation_role_swap.py`). E0–E4 has no slot for “claims to work, measurably does nothing”: it audits pairwise specification difference, whereas the escape is a claim-to-measurement gap. What catches it is the nullity check, not the taxonomy.

That account rests on two raters who share a provider, so we put the same question to a second provider, Atria-Dawn-Preview at temperature 0, shown payloads carrying the published raters’ own prompt digests so prompt drift cannot explain any agreement between them. It reproduces the blind spot rather than breaking it. On the six measured cases it recovers 3 of 6 verdict labels and 1 of 6 error-code labels, and the three verdicts it misses are exactly the three escapes, each returned as `not_equivalent/E4`. On the 36 real-source pairs they agree completely where a label exists: on the 33 labellable pairs all three reproduce the recorded verdict (33/33 each) and separate only on the error codes, matching the recorded set on 7, 9 and 6 of those same 33, with 26 of the 27 second-provider misses strict supersets of it. On the three unlabellable pairs every rater answers equivalent: none abstains where the rubric derives nothing. Agreement about *which* code fired, not about equivalence, is the scarce quantity. Switching providers buys coverage of a different failure mode, not freedom from this one. `scripts/rater_rollup.py` re-derives every number here from the per-case labels and refuses the artifact on drift.

3.6 A third substrate buys agreement, not independence

We also ran an architecturally distinct rater: a 421M ModernBERT *discriminative*, non-autoregressive head, which cannot generate the rationale an audit prompt asks for. It reaches 0.5 of 10 specification and 0.667 of 6 measured cases but **0 of 33** real generated pairs, answering equivalent on 35 of 36, and agrees with the two generative raters at $\kappa = 0.138$, -0.364 and -0.057 : on two of three sets it is anti-correlated with them. On the escape case it answers equivalent, which is *correct*, at a top-1 minus top-2 margin of 0.010: right for no reason a verifier can detect. Changing substrate does not buy independence, which is a property of the *evidence channel*, and only the simulator provides one here. Substituting it for the missing rater was the obvious move and not one we can make honestly, so the measured set stops at six cases.

4 What this should change

Each rule is a check an author can run; all eight were violated by us. (1) **Report factor levels beside every variance statistic**: a spread over fewer than two levels of its own axis is *unexercised* and can never satisfy a threshold. (2) **Treat labels as generated artifacts**: ground a verdict in measurement where you can, abstain where you cannot, and fuzz structural labelers against preprocessor lines and comments. (3) **Key identity on the delivered digest**,

and report the label marginal with every agreement statistic: $\kappa = 1.0$ on a single-class set means nothing, and a factor level whose design is another level’s design measures nothing however consistent its bookkeeping. (4) **Do not let a learned score decide what needs approval** unless it is shown to separate the safe action from the destructive one; a rule that escalates everything automates nothing. (5) **Make nullity a gate, not a diagnostic**: “compiles, runs, produces numbers” passed three designs that do nothing, and two of them equal the no-op reference on every trace. (6) **Record a source digest beside every measurement**: a binary digest says a build happened, not which source produced it, and a module name says less. (7) **Fail closed when a factor level has no artifact**: substituting a nearby candidate to keep a grid rectangular destroys the axis both ways and leaves no trace in the labels. (8) **Re-derive, do not re-run**: every repair here was scored against replies and measurements already on disk, so a label or attribution defect cost no simulator run and no new model query.

5 Limitations

Each grid is one design set. The 2×2 rectangle shows the seed and prompt effects surviving simultaneous variation, but the same code reports $\tau = 0.556$ then 0.778 and 78.16% then 23.50% cross-seed CV, so every magnitude here is a statement about a specific set of generated designs. **Repeats**: 0.0000% is $N = 5$ per cell (45 runs) and proves only determinism. **Labels**: the 33-pair κ rests on structural labels that survived our own header-parsing defect only because we could re-derive them, and no pair in that set is gold-labelled equivalent, so its verdict-axis agreement is degenerate rather than reliable. **Annotators**: the two generative raters the published κ rests on share one provider; the second provider reproduces the shared error on the measured and canonical cases but not on the 33-pair set; the one architecturally different rater we could reach is worse than independent (§3.6); and no rater here is a human. **Validity**: we tested role order (0/8 verdict flips) and not training-data contamination, which systematic reviews treat as the dominant risk [6]. **A re-ask is disclosed**: the second provider’s 10-of-10 canonical verdict score is a retry’s, since the first request returned no content at all and the retry, shown an identical prompt digest, parsed. **Provenance**: three historical unknown commits are left unrepaired, because writing a plausible SHA into a file that never measured one would be the defect itself. **Scope**: one simulator, one design family, three traces; no RTL flow and no gem5 result is claimed.

6 Conclusion

We released an audit loop that separates four variance sources, gates publication on measurement *and* annotation, and keeps its evidence. Against the official CHIA ChampSim image it blocked its own reproducibility claim at 78.16% cross-seed CV, showed seeds moving artifacts without moving designs, prompt response at 249.18% , and two same-provider auditors agreeing and both wrong on a measured simulator escape that a second provider repeats on byte-identical prompts. It also caught ten silent defects in our own automation, including a labeler that read `#include` brackets as comparison operators while the annotators it scored were right.

Defect	What it wrongly reported	Evidence that it was wrong	Now
1 Unexercised axis	cross-seed CV 0.0000% from one seed level, hence REPRODUCIBLE/PUBLISHABLE	the axis had one level; nothing was varied	levels counted; axis_not_exercised:seed blocks; grids rescored
2 Generated gold labels	three rename-only pairs labelled not_equivalent/E2, scored as shared rater errors	#include <vector> parsed as two comparison operators	relabel_semantic_cases.py, four regression tests, κ re-derived
3 Nullity scope	one flag for "inert here" and "inert everywhere"	s2 is null on imagick and 10.6% faster on BFSCC	null_on_every_measured_trace split, four tests
4 Unattributable rows	three incremental rows naming three modules on one binary_sha256 and one cycle count	digest and count are shared, so no row identifies a design	no claim rests on them; footnote in the report
5 Misattributed source	an escape case citing a file that cannot compile; the measurement came from a same-named file elsewhere	invalid static_cast from champsim::address to int64_t; 63 name collisions across 11 module names	audit_case_attribution.py (80 assertions), CANDIDATES.md index
6 Approval score	a needs-approval head escalating 6 of 6, local rewrite 0.895 above public push 0.848	Spearman ρ = +0.406 over 10 tiers, min(tier 3) – max(tier 0) = –0.169; no threshold separates safe from destructive	blast radius declared per option in code; score advisory
7 Self-filling harness	cross-seed CV 8.3812% comparing two designs under seed labels, 0.0% from measuring one binary twice	three of six levels fell back to gen_default_s0, and design_sha256 shows 3 designs behind 6 levels	harness raises on a missing level; audit_grid_levels.py re-checks every published grid
8 Fill value as measurement	top-1 stability 0.0 and τ = 0.0 on a one-candidate grid, printed as the most unstable ranking, gate PASS	one candidate cannot be ranked; the value was a fill	unrankable (fewer_than_two_candidates); a test asserts the printed form
9 Truthy pin	commit_sha_retained passing on the literal "unknown"	3 of 7 raw.json and 4 of 7 env_pin.json carry it; the fixture held "abc"	hex 7–40 chars required; three historical values left unknown and now fail
10 Asserted difference	classify returning not_equivalent for any text difference with no derivable error class	3 of 36 pairs have no shared trace measurement and no code	explicit unlabelable verdict, scored as no evidence

Table 1: Ten defects in our own automation, each found by reading primary evidence rather than by a failing gate. Five are one failure mode: a quantity whose name claims a measurement stops referring to the thing measured and still returns a value that reads as a result. Defect 7 alone manufactures its evidence rather than misreading it, and it is where the first real positive control for the nullity branch came from: two designs equal the no-op reference *exactly* on all three traces (2,261,770 / 7,183,123 / 1,561,323 cycles) on binaries that are not the image default.

Metric	Gate	Measured
Cross-seed CV (max)	< 5%	78.16% / 23.50%
Repeated-run CV (max)	report	0.0000%
Prompt spread (max)	report	249.18% / 179.67%
Trace CV (max)	report	163.33%
Trace top-1 stability	report	0.667 / 1.000
Mean Kendall τ	report	0.556 / 0.778
κ , spec cases	≥ 0.7	0.859
κ , real pairs (33 label.)	≥ 0.7	0.636
κ , verdict only [†]	≥ 0.7	1.000
κ , 6 measured	≥ 0.7	0.455
Verdict acc., 6 measured	≥ 0.7	0.500
Second-provider verdict/code	report	3/6 / 1/6

Publish gate both pass BLOCKED

Table 2: Pre-registered gates against measured values; where a row carries two numbers, the second is the 2×2 rectangle. [†]degenerate: all 33 labellable gold labels in that set are not_equivalent, so a constant rater also reaches 1.0 and the combined-label row is the informative one. Third-substrate κ is the discriminative head against the two generative raters on the spec / measured / real-pair sets in turn.

The instrument, the negative verdicts and the self-reports are the contribution.

AI assistance in this work

AI coding and writing tools assisted with implementation, experiment plumbing, and the drafting and editing of this paper. An AI agent ran the audit loop, wrote and revised the scripts and the evidence pipeline, and drafted this text under the author’s direction; the author selected the claims, checked them against the primary artifacts, and is responsible for all content. The evaluation layer is machine-generated as well: two generative raters from one provider, a third from a second provider (Atria-Dawn-Preview, api.atria-asi.ai, queried 2026-09-24 at temperature 0, per-case prompt digests in the artifact), a generative labeler shown to be wrong while the raters were right (Table 1 row 2), a local

discriminative head used as both third annotation substrate and decision gate (§3.6, Table 1 rows 6), and ten self-audits by the same agent that produced the code it reviewed. A generated object counts as evidence only with its producer, version and failure modes attached; the negative verdicts here came from a process that was not independently audited, so the repository is the thing to check.

References

- [1] R. Gupta et al. ArchAgent: Agentic AI-driven Computer Architecture Discovery. arXiv:2602.22425, 2026.
- [2] A. Cui, F. Hermida-Rivera, J. Toubes et al. CHIA: An open-source framework for principled, agentic AI-driven hardware/software co-design research. arXiv:2606.27350, 2026. <https://github.com/ucb-bar/chia>.
- [3] Meerkat Research. Finding Widespread Cheating on Popular Agent Benchmarks. <https://debugml.github.io/cheating-agents/>, 2026.
- [4] OpenAI. Why SWE-bench Verified no longer measures frontier coding ability, 2026. <https://openai.com/index/why-we-no-longer-evaluate-swe-bench-verified/>; and OpenAI, An audit of SWE-Bench Pro, 2026.
- [5] J. Wang, F. Bianchi, S. Zhu et al. Automated Benchmark Auditing for AI Agents and Large Language Models. arXiv:2605.26079, 2026.
- [6] E. Nourbakhsh, M. S. Sirjani, A. Mousavi et al. Are LLM Benchmarks Already Contaminated? A Systematic Review of Contamination Detection Methods. Proc. 5th Generation, Evaluation and Metrics (GEM) Workshop, Outstanding Paper. <https://aclanthology.org/2026.gem-main.50/>, 2026.